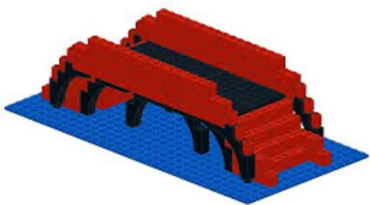




Comp 2140 Review



What we have learnt

- Generate scanner and parser
 - We do not program directly
 - Instead we write the specifications for the scanner and parser
- Describe specification using (formal) grammar
 - Grammar for scanner is simpler (regular grammar)
 - Grammar for parser is more complex (CFG)
 - Programming languages are defined using BNF and EBNF
- Understand how grammar is translated into program
 - RE → NFA → DFA → Minimization
 - CFG → LALR Diagram → Shift/reduce or reduce/reduce conflict
- We can write the assignments
- We can write the grammars
- We can debug, and write a similar tool in the future

Higher level skills

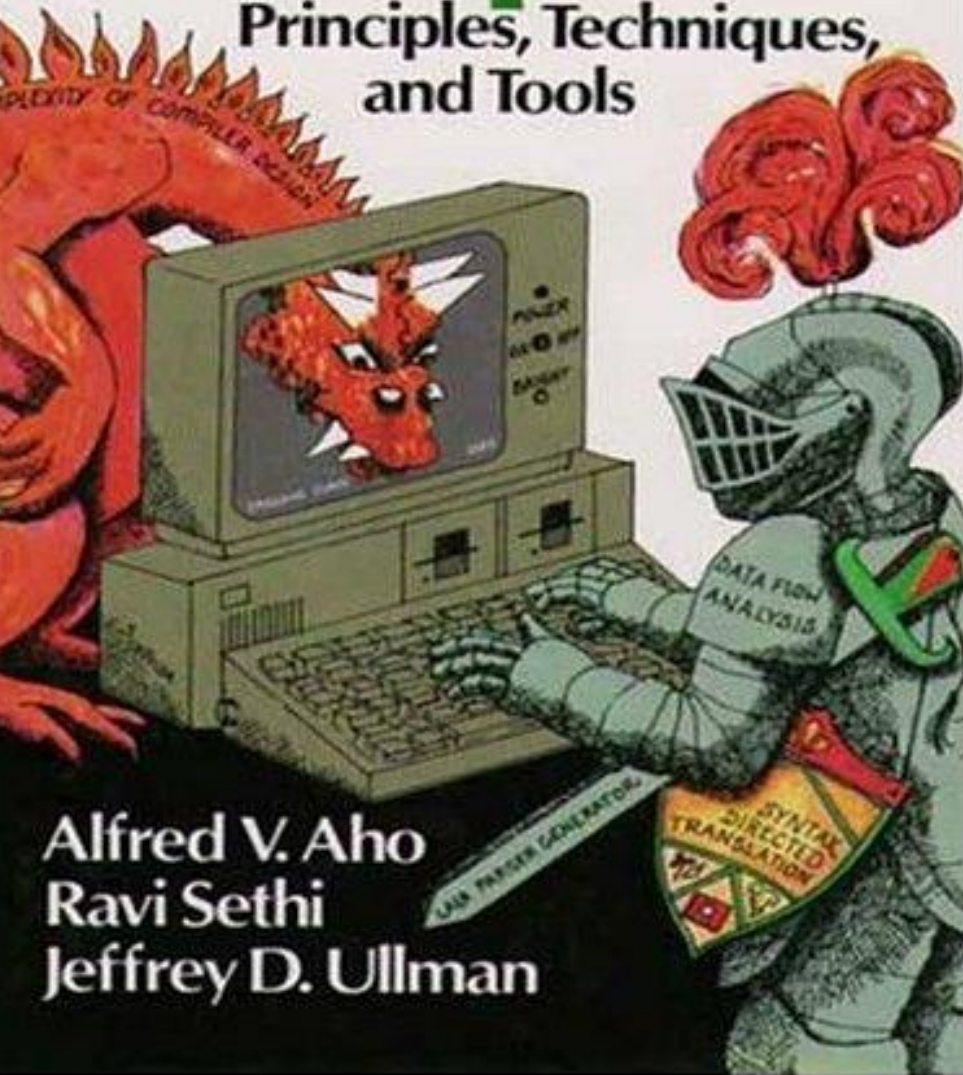
- Deal with large problems
 - Formalize the problem
 - Divide the problem into smaller ones
 - Compiler $\rightarrow$ scanner $\rightarrow$ RE $\rightarrow$ NFA $\rightarrow$ DFA
 - Find an algorithm to solve the problem
 - RE $\rightarrow$ NFA $\rightarrow$ DFA ...
- Develop a generic solution for a wide range of problems
 - Generate a parser for any language
 - Guarantee the solution is always correct
 - Repetitive code is always saved

What makes a good programmer (from c2.com)

- "We will encourage you to develop the three great virtues of a programmer: *laziness, impatience, and hubris*." -- [Larry Wall](#), [ProgrammingPerl](#), [OreillyAndAssociates](#)
- **Laziness**
 - The quality that makes you go to great effort to reduce overall energy expenditure. It makes you write labor-saving programs that other people will find useful, and document what you wrote so you don't have to answer so many questions about it.
- **Impatience**
 - The anger you feel when the computer is being lazy. This makes you write programs that don't just react to your needs, but actually anticipate them.
- **Hubris**
 - Excessive pride. Also the quality that makes you write (and maintain) programs that other people won't want to say bad things about.

Compilers

Principles, Techniques,
and Tools

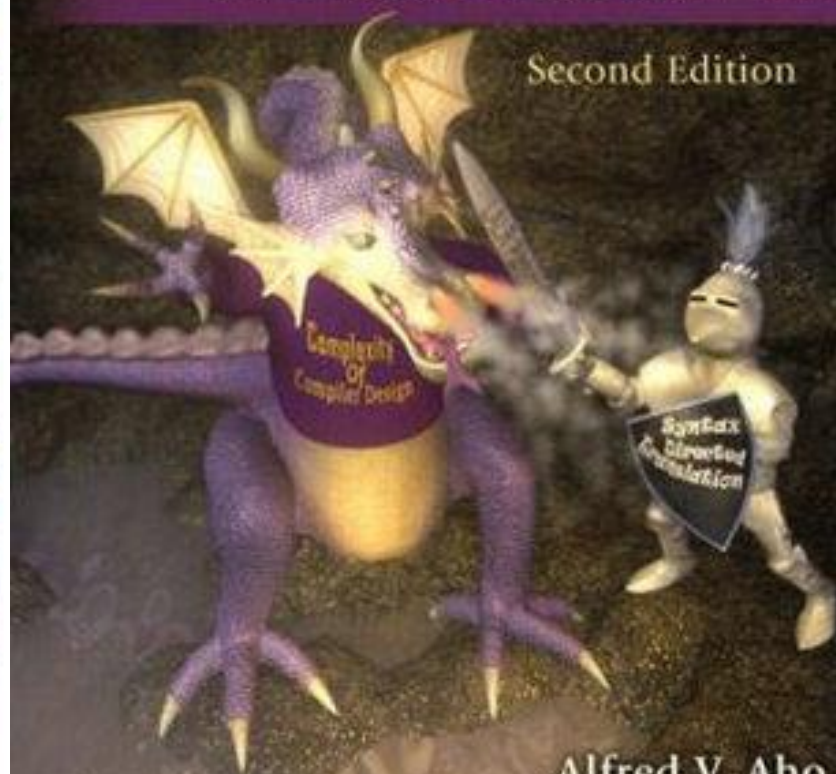


Alfred V. Aho
Ravi Sethi
Jeffrey D. Ullman

Compilers

Principles, Techniques, & Tools

Second Edition



Alfred V. Aho
Monica S. Lam
Ravi Sethi
Jeffrey D. Ullman

Valid topics

- Anything that was mentioned in the lectures
 - Also check lecture slides
- Assignments will be tested

Important topics

- Lexing
 - RE, NFA, DFA
 - RE to NFA, NFA to DFA, DFA minimization
- Parsing
 - CFG
 - LL parsing
 - LR parsing
- Understand grammar
- Write a grammar
- Write a parser or translator
- Understand how parser works
 - Shift/reduce conflicts

Lexing

- What is lexing? what is a lexer?
- How does a lexer relate to NFA/DFA theory?
- How does a lexer fit in with the rest of a compiler?
- What is a regular language?
- How do you write a regular expression, based on a narrative description of the pattern?
- How do you make an NFA based on an RE?
- How is an NFA different from a DFA?

Parsing

- What is a context-free grammar?
- What is the grammar hierarchy?
- What is parsing? What is a parser?
- How does a parser relate to CFG theory?
- What is a leftmost derivation and rightmost derivation?
- What is a parse tree?
- What is ambiguity? How to remove ambiguity?

LL parsing

- What is FIRST()?
- What is FOLLOWS()?
- How do you fix left recursion?
- How do you fix common prefixes?
- How do you build a parse table?
- How do you run an LL parser?

LR parsing

- What is a shift/reduce conflict?
- How do you fix a shift/reduce conflict?
- What is LR(0) configuration (item)? What is LR(1) item?
- What is CLOSURE()?
- What is Successor(S, A)?
- How to draw transition diagram for LR(0), SLR, LR(1)?
- How to construct parsing table for LR(0), SLR, LR(1)?
- How to run LR(0)/SLR/LR(1) parser?
- How to decide whether a grammar is LR(0)/SLR/LR(1)?
- What is the difference between LR(0), SLR, LR(1) and LALR?
- Which LR algorithm does javaCUP, yacc use?

LL(1)

Given the grammar

$$A \rightarrow aA \mid bA \mid b$$

1. Whether is it an LL(1) grammar? Why?
2. If not, can you change that to an LL(1) grammar?

Answer:

It is not an LL(1) grammar,
because there is a conflict in
the LL(1) parse table

	a	b	\$
A	$A \rightarrow aA$	$A \rightarrow bA$ $A \rightarrow b$	

Modified grammar:

$$A \rightarrow aA \mid bA'$$

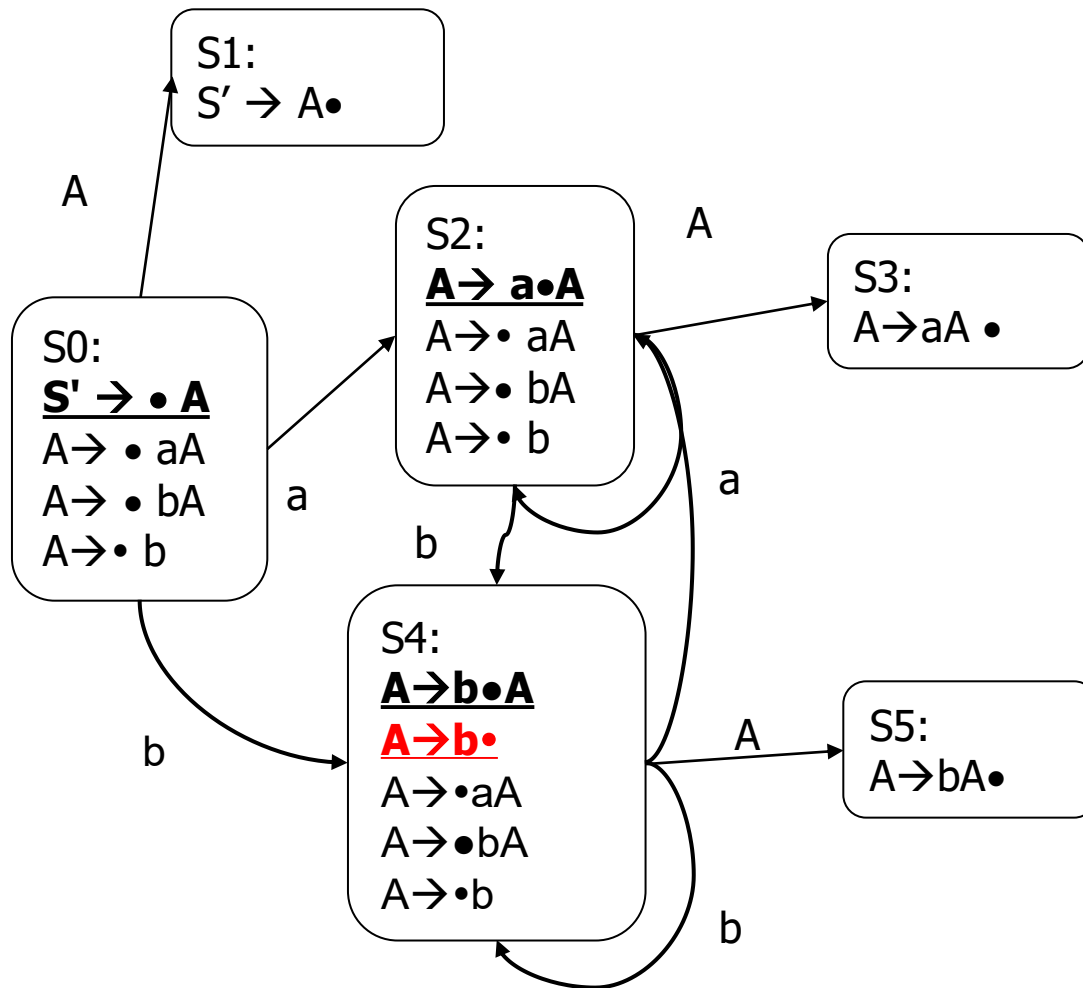
$$A' \rightarrow A \mid \epsilon$$

	a	b	\$
A	$A \rightarrow aA$	$A \rightarrow bA'$	
A'	$A' \rightarrow A$	$A' \rightarrow A$	$A' \rightarrow \epsilon$

Is the grammar LR(0)?

$A \rightarrow aA \mid bA \mid b$

It is not LR(0) because there is a conflict in state S4

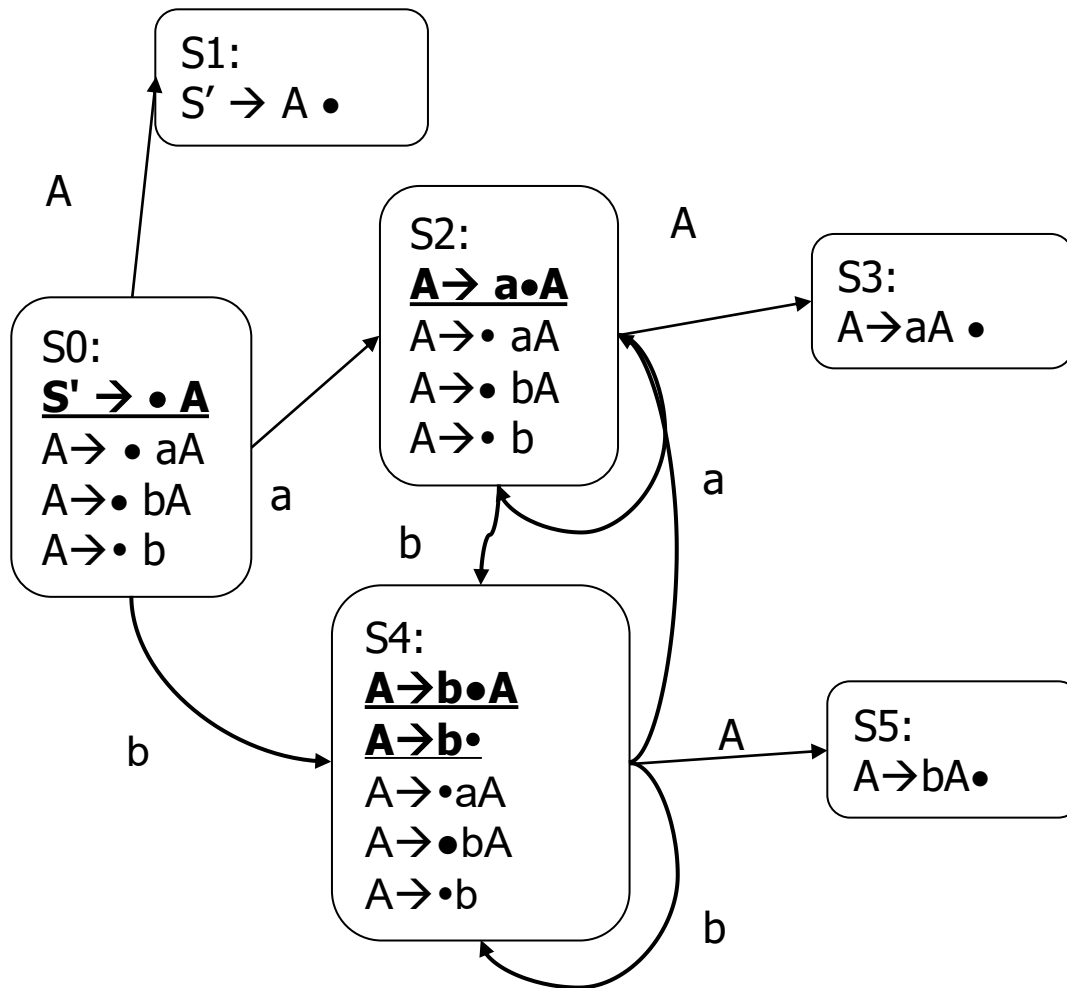


Stack	Input	action
S0	aab\$	S2
S0 S2a	ab\$	S2
S0 S2a S2a	b\$	S4
S0 S2a S2a S4b	\$	R $A \rightarrow b$
S0 S2a S2a S3A	\$	R $A \rightarrow aA$
S0 S2a S3A	\$	R $A \rightarrow aA$
S0 S1	\$	accept

Is the grammar LR(0)?

$A \rightarrow aA \mid bA \mid b$

It is not LR(0) because
there is a conflict in
state S4



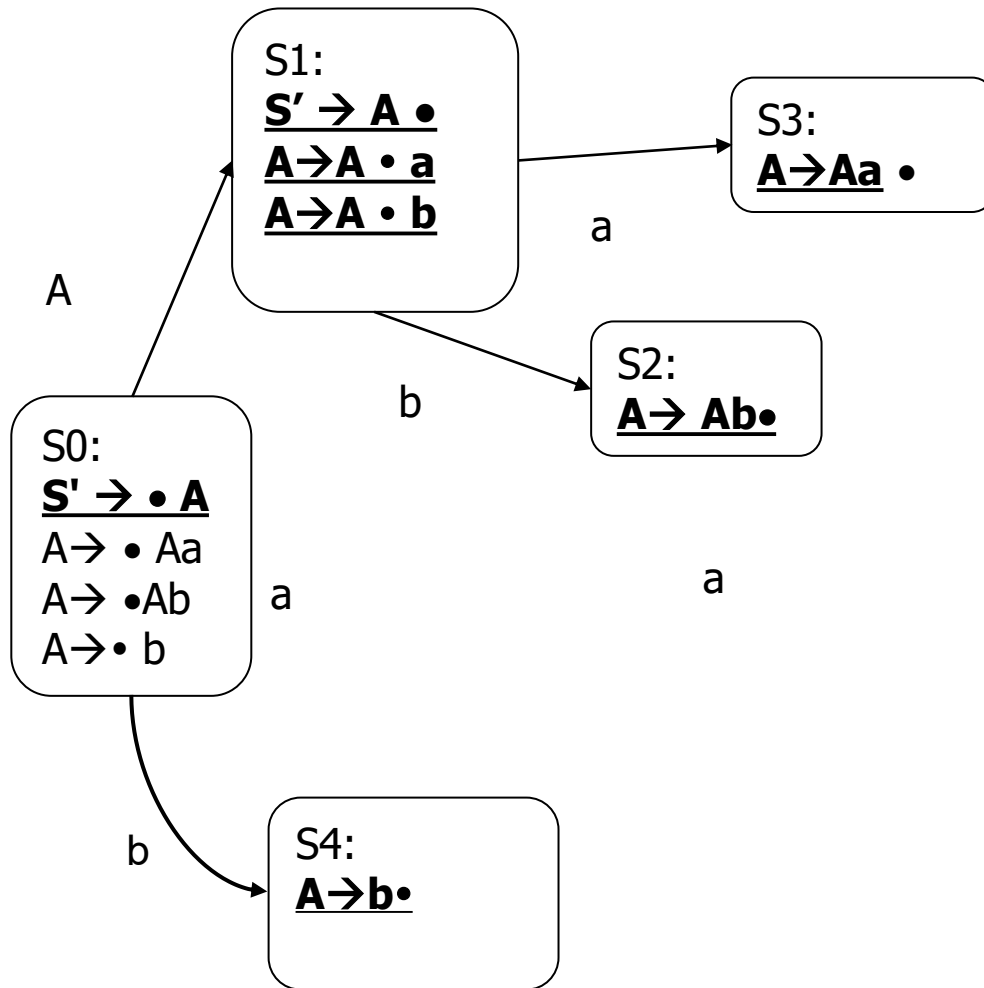
Stack	Input	action
S0	abb\$	S2
S0 S2a	bb\$	S4
S0 S2a S4b	b\$	S4 or Reduce?
S0 S2a S4b	b \$	If R, $A \rightarrow b$
S0 S2a S3A	b \$	R $A \rightarrow aA$
S0 S1A	b \$	?

Whether it is LR(0)?

$A \rightarrow Aa$

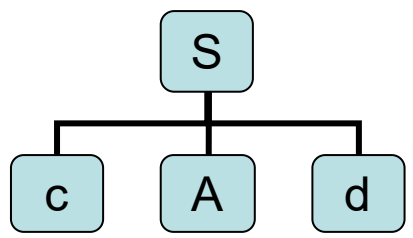
$A \rightarrow Ab$

$A \rightarrow b$

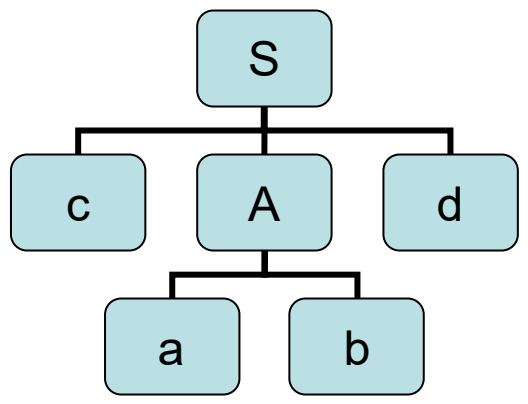


Stack	Input	action
S0	bab\$	S4
S0S4	ab\$	R $A \rightarrow b$
S0S1	ab\$	S3
S0S1S3	b \$	R $A \rightarrow Aa$
S0S1	b \$	S2
S0 S1S2	\$	R $A \rightarrow Ab$
S0S1	\$	Accept

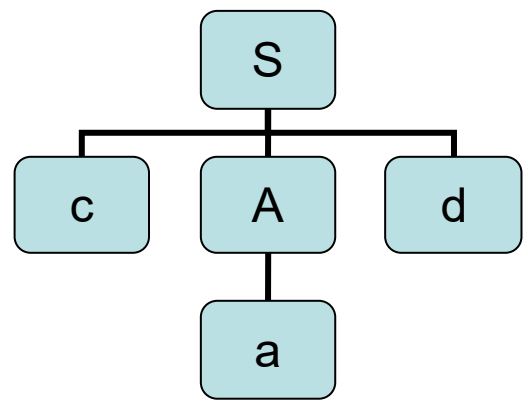
- $S \rightarrow cAd$
- $A \rightarrow ab|a$
- $w=cad$



cad
↑



cad cad
↑ ↑



cad
↑

Is the grammar LL(1)?

$S' \rightarrow S$

$S \rightarrow cAd$

$A \rightarrow ab|a$

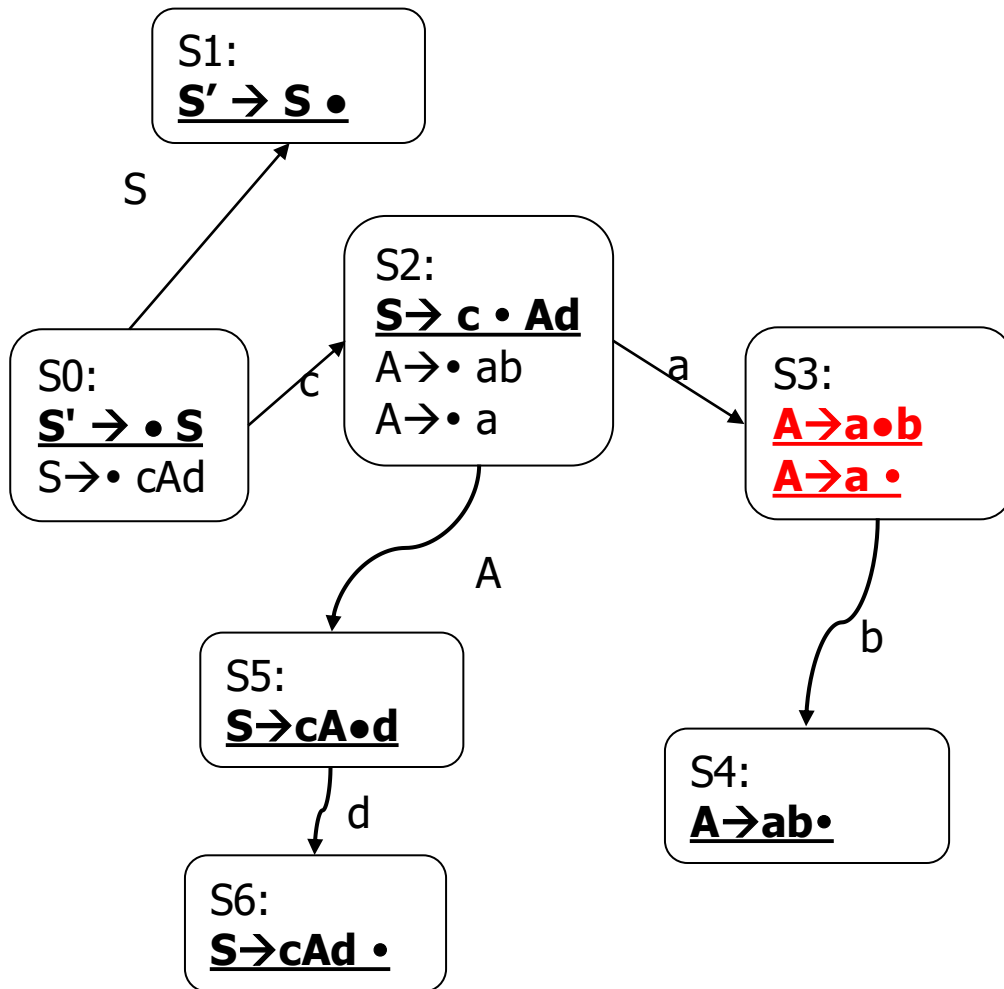
It is not LL(1)

1. because the LL(1) parsing table has conflict; OR
2. Because it is not left factored

	a	b	c	d	\$
S'			$S' \rightarrow S$		
S			$S \rightarrow cAd$		
A	$A \rightarrow ab$ $A \rightarrow a$				

Is it LR(0)? SLR?

$S' \rightarrow S$
 $S \rightarrow cAd$
 $A \rightarrow ab|a$



Stack	Input	action
S0	cad\$	S2
S0 S2c	ad\$	S3
S0 S2c S3a	d\$	Reduce not Shift
S0 S2c S5A	d \$	S6
S0 S2c S5A S6d	\$	R $S \rightarrow cAd$
S0 S1	\$	Accept

Sample questions

- JLex specification defines a
 - Context Free Grammar;
 - Regular Grammar;
 - Context Sensitive Grammar;
 - None of the above.
- JLex specification has _____ parts, separated by %%.
 - Two;
 - Three;
 - Four;
 - Five;
 - None of the above.

- JLex does not deal with:
 - DFA minimization;
 - CFG;
 - NFA to DFA transformation;
 - Lexical analysis;
 - None of the above.
- A JavaCup specification defines a
 - Context Free Grammar;
 - Regular Grammar;
 - Context Sensitive Grammar;
 - None of the above.

- Suppose that you have a grammar that can give two different derivations for the same sentence. Is that grammar ambiguous?
 - Definitely yes;
 - Definitely no;
 - There is not enough information to tell;
 - It can't have two derivations;
 - None of the above.

- With an ambiguous grammar, how many parse trees are there for any sentence that is **not** in the language?
 - 0;
 - exactly 1;
 - more than 1;
 - 1 or more;
 - None of the above.

- Given a grammar that contains the following production rule, where **A** is a nonterminal and **a** and **b** are terminals:

$A \rightarrow aAa \mid abba$

According to Chomsky hierarchy, the grammar is in

- Level 0;
- Level 1;
- Level 2;
- Level 3.
- None of the above.

- Which of the following is not involved in compiler construction:
 - Lexical analysis;
 - Linear analysis;
 - Code generation;
 - Semantic analysis;
 - None of the above.

- Given the following rules of a grammar, where **A** and **B** are non-terminals, **a** and **b** are terminals, and **A** is the start symbol:

$$A \rightarrow aB \mid bB$$

$$B \rightarrow aB \mid bB$$

Which of the following regular expression can recognize the same language?

- $(a \mid b)^+$
- $(a \mid b)^+abb$
- $(a \mid b)(a \mid b)^+$
- $ab(a \mid b)^+$
- None of the above.

Answer true or false for the following questions:

- $(0|1)^* = ((1|0)^*)^*$
- For every language, there is an unambiguous grammar.
- JLex is used to generate a parser from a JLex specification.
- Consider the following grammar where **S** is a non-terminal, **if**, **then**, and **else** are terminals.

$S \rightarrow \text{if then} \mid \text{if then else} \mid \epsilon$

Whether the grammar is ambiguous?.

- YACC is a parser generator.
- Top down parsing method has the name because it scans input file from top to down.

- In the following grammars **E** is a non terminal and **ID** is a terminal.

- Remove the left recursion of the following grammar.

$$E \rightarrow E+ID \mid ID$$

- Write the result of the left factoring of the following grammar

$$E \rightarrow ID+E \mid ID$$

Some solutions not so good

- swap E and ID

$$E \rightarrow ID + E \mid ID$$

- Indirect left recursion

$$E \rightarrow E' + ID \mid ID$$

$$E' \rightarrow E$$

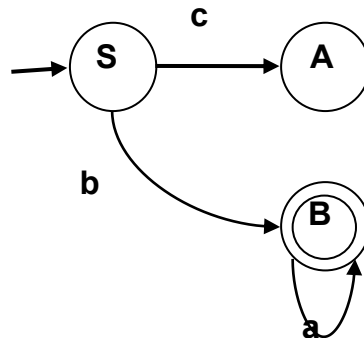
$$E \rightarrow A \mid ID$$

$$A \rightarrow E + ID$$

Acronyms

- FSM
- NFA/DFA
- BNF
- LL
- LR
- LALR
- ...

- Given the following transition diagram. Write the corresponding regular expression.



- Given the following grammar, where **A** is a non-terminal, **a** and **b** are terminals:

$$A \rightarrow aA \mid bA \mid b$$

Write the regular expression that can recognize the same language.

- Given the regular expression **(ab)***. Write the corresponding **regular** grammar. Note that you will not get any marks if the grammar is not regular.
- Some incorrect answers:
 - $A \rightarrow abA \mid \varepsilon$
 - $A \rightarrow BA \mid \varepsilon$
 - $B \rightarrow ab$

- Write a CFG for the following languages over alphabet $\{a, b\}$:
 - Palindromes, i.e., strings read the same backward and forward, such as “aaa”, “aabbabbaa”.

- Given the following production rule, where **E** is a non-terminal, and **identifier** is a terminal. Is it an ambiguous grammar? Explain your conclusion.

$E \rightarrow E * E \mid \text{identifier}$

- Rewrite the grammar into an unambiguous one

- Given the following grammar

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid id$

- What are the values in $\text{First}(T)$?
- What are the values in $\text{Follow}(T)$?